Evidence Ledger

FactBank: Trails and Evidence Ledger

Where every claim in the report maps to an actual file in the repository

By Mohannad. This ledger was drafted by an AI assistant, because I do not yet know how to write a formal research paper. It is the companion to the research report. For each claim, it points to the real file in the repository (paths are relative to the repository root; the canonical tree is v2/). Every file named here was checked to exist in the repository at the time of writing.

1. How this ledger is organised

This ledger exists so that anyone can open the exact file behind any number or claim in the report. Every row below names a real path in the repository and says what that file holds. The rule is the same one used throughout the project: the count only says what to read, and reading the file is the verdict. Paths use forward slashes and are relative to the repository root; on my machine the root is the connected project folder. The canonical, latest-only tree is v2/, and the historical evidence trail lives under archive/.

Convention: where a file name contains a version like template-brain-v3.1 or a size like BAKE-TEST-REPORT-12b, that is the literal file name in the repository, not a typo.

2. The numbered finding trail (F-001 to F-065)

The core evidence trail is a single numbered log. It is the proof base behind every F-number cited in the report.

File	What it holds
archive/docs/FINDINGS.md	The full F-001 to F-065 log: each finding is a measured result, an invalidated measurement, or a discovered capability, in chronological order with an index table at the top.
archive/docs/RESULTS.md	Consolidated results that back several findings.
archive/docs/PROGRESS.md	The running progress narrative behind the trail.
CLAUDE.md (repo root)	The working guidance for the repo; summarises the hard rules and points at where each F-number's proof lives.

Key findings cited, and the shortest description of each: F-040 (the whole-bank static prompt was rejected for serving, roughly 6k-token tax), F-046 (control culling), F-050 (jinja2 is not the shipped engine), F-051 (inverted index beats a linear scan), F-053 (the LM Studio silent sentinel swap), F-056 (precision-at-density is the wall), F-059 (the size wall is beaten via model.yaml),

F-063 (live obedience 8/8 vs 3/8 bare), F-064 (66% of the mined pool is junk), F-065 (signature-mined facts win zero cases). All are entries inside archive/docs/FINDINGS.md.

3. Thesis, architecture, and top-level

Claim in the report	Evidence file(s)
The thesis, the public model table, and base to baked headline numbers	README.md (repo root)
The three-way architecture split (Baked-Index shipping, Static-Bake legacy, Served-Loop test bench)	v2/ARCHITECTURES.md
The A-to-Z build playbook (pick library, mine, extract, bake, serve, test)	v2/BUILD-AN-EXPERT.md
The v2 map and current state	v2/README.md
The static-bake code (legacy route)	v2/package/factbank/bake.py
The Python test bench (retrieval prototype, no baking)	v2/extractor/lookup.py

4. Fact-extraction pipeline

Claim in the report	Evidence file(s)
FIND, extract, repair, check pipeline and the one law	v2/extractor/BLUEPRINT.md; v2/extractor/EXTRACTOR-SPEC.md
The extract stage (function-calling, deepseek-v4-flash, chunk 3500, code-derived fields)	v2/extractor/extract.py
The repair stage (snap paraphrased quote to a real line)	v2/extractor/repair.py
The check stage (verbatim-quote anchor, field validation, .rejects)	v2/extractor/check.py
The driver that chains the three stages	v2/extractor/run.py
The fact schema and field definitions	v2/extractor/SCHEMA.md
The 2.0 method (code in the fact, one appsec_core pipeline, sentence-boundary repair, adversarial audit)	v2/extractor/EXTRACTOR-2.0.md
The design record (what worked, what was rejected)	v2/extractor/BLUEPRINT.md
k8s 35 to 96.5%, pydantic 81.5 to 97.8%, +901 facts from 3 checker bugs, the 58% that hid ~51 good facts, reachability 69.6%	v2/extractor/PROGRESS.md
OpenAI-compatible endpoint switch (LLM_BASE_URL, LLM_MODEL, LLM_API_KEY)	v2/extractor/extract.py; v2/extractor/README.md
Currency vs grounding, the ldap3 SYNC to RESTARTABLE reversion	v2/extractor/BLUEPRINT.md

Duplicate-id repair and paraphrase dedup	v2/extractor/fix_dup_ids.py; v2/extractor/dedupe.py
The evidence-only concrete-rewrite pass (not in the pipeline)	v2/extractor/refine.py

5. The bank and its provenance

Claim in the report	Evidence file(s)
Storage layout, curated vs mined, the 66% junk and 24,133 to 8,096 gating	v2/serving/ARCHITECTURE.md; v2/bake/template-brain-v3.1/dedupe_mine.py
The mined pool builder (inspect.signature over installed libs)	v2/bake/template-brain-v3.1/mine_api.py; v2/bake/template-brain-v3.1/build_pool.py
Landmine-only policy and the domain value function	v2/extractor/experts/DEPARTMENTS.md
Sharpened value function (low training representation, bundling law)	v2/extractor/experts/NEXT-EXPERTS-STUDY.md
Per-expert coverage (libraries, fact counts, status)	v2/extractor/experts/*/COVERAGE.md (ai-ml, web, devops, gitchameleon, security-networking)
The faceted v3 schema (concept to variant, feature phrases, benign-prompt bridge)	v2/decisions/SCHEMA-V3.md
The shipped appsec bank (258 concepts / 254 CWE, 3,984 variants, 1,075 with code)	v2/extractor/experts/appsec/facts/FINAL_v3.jsonl

6. Baking: the in-template inverted index

Claim in the report	Evidence file(s)
Term weights, squash-normalisation, Doc2Token filter, IDF buckets, DF cap, control culling, determinism, RAWGGUF_CAP	v2/bake/template-brain-v3.1/bake_index.py
Keyword extraction, COMMON_DF, stopwords, control normalisation	v2/bake/template-brain-v3.1/enrich.py
Index-design constants and the D-series prototypes	v2/bake/template-brain-v3.1/jinja_lab/designs.py
GGUF write-back, GATE_ALIASES manual table, write_baked	v2/bake/template-brain-v3.1/bake_template_v3.py
The gate-alias fix (every rename's old name becomes an alias)	v2/bake/template-brain-v3.1/gen_gate_aliases.py
Offline proof the gate injects (Volatility 3: 0 to 5 facts)	v2/bake/template-brain-v3.1/render_retrieval.py
Render verifier and parity	v2/bake/template-brain-v3.1/verify_render.py; v2/bake/template-brain-v3.1/parity.py

Bank-to-index adapters (per department)	v2/bake/template-brain-v3.1/adapt_gc.py; adapt_secnet.py; adapt_expert.py
Size-wall trimmer	v2/bake/template-brain-v3.1/select_facts.py
LM Studio applied-bytes checker (the 48-char sentinel)	v2/bake/template-brain-v3.1/ lmstudio_yaml_test/check_override.py
Index stats 17,260 terms / 34,671 postings and template bytes	v2/serving/ARCHITECTURE.md; v2/serving/RESULTS.md
The thinking-ON template bases (channel-open fix, forced enable_thinking)	v2/bake/template-brain-v3.1/family_bases/ gemma4_think.jinja; gemma4_think.source.jinja

7. Runtime retrieval and delivery (the Jinja template)

Claim in the report	Evidence file(s)
The bound data (fb_post, fb_txt, fb_lib, FB_NCUR, FB_MAX)	v2/bake/template-brain-v3.1/inserts/ gemma4_idx/fb_preloop.jinja
The 7-step retriever, curated +6 / +12 scoring, top-5, the forged tool call	v2/bake/template-brain-v3.1/inserts/ gemma4_idx/fb_gen.jinja
The real tool-lane variant (when a client declares factbank_search)	v2/bake/template-brain-v3.1/inserts/ gemma4_idx/fb_toolmsg.jinja
The system canary and menu; delivery placement	v2/bake/template-brain-v3.1/inserts/ gemma4_idx/fb_sys.jinja; fb_user.jinja; fb_hook.jinja
The no-op top (why thinking is not forced on)	v2/bake/template-brain-v3.1/inserts/ gemma4_idx/top.jinja
Prose vs tool-lane (8,189 tokens vs 390 to 757), caching, list vs dict	v2/serving/ARCHITECTURE.md; v2/serving/PAPER.md; v2/papers/template-brain-report.md

8. Serving, limits, and the size wall

Claim in the report	Evidence file(s)
Sampling settings, max_tokens shared budget, thinking behavior	v2/BUILD-AN-EXPERT.md; v2/serving/OPERATIONS.md
The size wall (980,000 B), the sentinel, model.yaml route at 1.5 and 2.0 MB	v2/serving/LIMITS.md; v2/serving/SHIPPING.md
Index vs scanner (545 vs 2,629 ms), USES vs CODE, gate results	v2/serving/RESULTS.md; v2/serving/RETRIEVAL-V7.md
The three Jinja engines and the F-number prose (F-042, F-050, F-053, F-059)	v2/decisions/LANGUAGES.md; v2/papers/template-brain-report.md
The serve launcher (llama-server with --jinja, 6-parallel)	v2/bake/serve_factbank.ps1

9. GitChameleon 2.0 evaluation

Claim in the report	Evidence file(s)
12B 37.8 to 44.2%, 26B 43.4 to 46.2%, fixed/broke, all-328, thinking on/off, two-pass 54.2%, empties 30.2%	v2/eval/gitchameleon/BAKE-REPORT.md
Frontier leaderboard table and the comparability caveats	v2/eval/gitchameleon/LEADERBOARD-COMPARISON.md
Benchmark description, library counts, change-type histogram	v2/eval/gitchameleon/FACTBANK-NOTES.md; v2/eval/gitchameleon/README.md; v2/eval/gitchameleon/dataset/README.md
Vendoring, pinned commit, license, arXiv id	v2/eval/gitchameleon/PROVENANCE.md
Bank coverage (4,167 facts, 23 doors, 316/328 sourced)	v2/extractor/experts/gitchameleon/COVERAGE.md
The execution scorer (uv venv, hidden pytest, 240s timeout)	v2/eval/gitchameleon/run_tests.py
The base-vs-baked driver over llama-server	v2/extractor/experts/gitchameleon/test_baked.py

10. Security-expert evaluation and transcripts (netsec, offsec, dataplane)

Claim in the report	Evidence file(s)
The 3-size curve (netsec e2b/12b/26b), error-closure, easy/hard split	v2/extractor/experts/security-networking/BAKE-TEST-CURVE-3MODEL.md
Per-model netsec reports (26b, 12b, e2b), auto vs hand 35 to 39	v2/extractor/experts/security-networking/BAKE-TEST-REPORT.md; BAKE-TEST-REPORT-12b.md; BAKE-TEST-REPORT-e2b.md
The shared methodology and model settings (thinking-on + authority, 6-parallel)	v2/extractor/experts/security-networking/METHODOLOGY.md; MODEL-SETTINGS.md
netsec coverage and libraries	v2/extractor/experts/security-networking/COVERAGE.md; README.md; RESEARCH.md
The netsec test harness and question sets	v2/extractor/experts/security-networking/test_secnet.py; test_questions.jsonl; test_questions_hard.jsonl; probe_vol.jsonl
Raw netsec transcripts and scores (per size, base and baked)	v2/extractor/experts/security-networking/runs/{base,baked}-{e2b,12b,26b}-*_transcript.jsonl and *_score.txt
offsec and dataplane facts, sources, and question sets	v2/extractor/experts/offensive-security-re/{facts,sources,test_questions.jsonl}; v2/extractor/experts/ebpf-dataplane/{facts,sources,test_questions.jsonl}
offsec and dataplane base to baked numbers and regressions (in the model cards)	v2/publish/offensive-security-re/*/README.md; v2/publish/ebpf-dataplane/*/README.md
The overnight two-expert run log and tech investigation	TEMP-run-results-2026-07-17.md (repo root)

11. The application-security expert (built, baked, shipped 2026-07-18/19)

A large application-security expert whose bank is insecure-by-default landmines: it makes the model write secure code without being asked (e.g. torch.load weights_only=True, a post-cutoff default; XXE resolve_entities=False/no_network; yaml.safe_load; secrets over random; parameterized SQL; ast.literal_eval over eval; constant-time HMAC). This is the fourth expert in the shipped model line (netsec, offsec, dataplane, and security/appsec).

11.1 The bank, schema, and provenance

Claim in the report	Evidence file(s)
The faceted v3 schema (concept to variant, feature_phrases, benign-prompt bridge)	v2/decisions/SCHEMA-V3.md
The shipped bank: 258 concepts (254 CWE + 4 synthetic door groups) to 3,984 variants, 1,075 with verbatim bad/good code, 10+ languages	v2/extractor/experts/appsec/facts/FINAL_v3.jsonl
The 2.0 mining/indexing/audit method (7 permissive sources, one appsec_core pipeline, code in the fact, adversarial audit that removed ~3.8%)	v2/extractor/EXTRACTOR-2.0.md

11.2 The thinking-ON enablement fix

Claim in the report	Evidence file(s)
The two-bug root cause (Gemma-4 generation-prompt channel bug + weak authority framing), the channel-open fix, forced enable_thinking, the ~10% fail-safe residual	v2/decisions/TICKET-thinking-on-enablement.md
The thinking-ON template bases (open the thought channel, enable_thinking as template default)	v2/bake/template-brain-v3.1/family_bases/gemma4_think.jinja; gemma4_think.source.jinja

11.3 Cross-model SecurityEval, on the SHIPPED baked GGUFs (thinking-off)

External SecurityEval (s2e-lab, MSR 2022, 121 Python CWE tasks), common pattern-judgeable 21-task subset, hand-scored. e2b 13 / 12b 17 / 26b 19 / DeepSeek-V4 14; 12B+bank and 26B+bank beat the cloud model.

Claim in the report	Evidence file(s)
The cross-model scorecard (e2b 13 / 12b 17 / 26b 19 / DeepSeek-V4 14 on the 21-task subset; per-size lift; XXE sweep)	v2/extractor/experts/appsec/benchmark/SCORECARD-crossmodel.md
Raw arms, per size, base and bank	v2/extractor/experts/appsec/benchmark/se_{e2b,12b,26b}_{base,bank}.jsonl
The DeepSeek-V4 no-thinking arm	v2/extractor/experts/appsec/benchmark/eval_securityeval_deepseek.jsonl

The served-loop e2b/SecurityEval scorecards (retrieval-METHOD CEILING, served, NOT the shipped artifact)	SUPERSEDED by 11.3 for shipped numbers: v2/extractor/experts/appsec/benchmark/SCORECARD-e2b.md; SCORECARD-securityeval.md

11.4 Publishing and delivery (the 4.18 MB template drove the model.yaml route)

The appsec bank template is 4.18 MB, over LM Studio's ~980 KB raw-GGUF cap (F-053), so it is the first expert to ship the LM Studio Hub model.yaml route.

Claim in the report	Evidence file(s)
The build/publish driver and the six-way bake	v2/publish/security-appsec/build_publish.py; v2/publish/security-appsec/bake6.py
The model-card writer	v2/publish/security-appsec/write_cards.py
The 3 Hugging Face GGUF cards (both editions, both templates, both model.yaml)	v2/publish/security-appsec/gemma-4-{E2B,12B,26B-A4B}-security-expert-GGUF/README.md
6 LM Studio Hub virtual models gemma-4-{e2b,12b,26b-a4b}-security-expert (+ -thinking); Information Security EXPERTS collection	v2/publish/security-appsec/build_publish.py; write_cards.py

12. Reasoning paradox and authority probes

Claim in the report	Evidence file(s)
The 18/18 framed authority result, the unlogged control, the 4B ladder, Qwen 31.3%	v2/eval/gitchameleon/QWEN-THINKING-AUTHORITY.md
The concrete thinking-ON fix (channel-open + strong authority + forced enable_thinking, the ~10% spiral residual)	v2/decisions/TICKET-thinking-on-enablement.md
The authority probe harness (conditions A/B/C, verdict regex)	v2/extractor/experts/gitchameleon/probe_qwen_authority.py
The retained 18-row authority log	v2/archive/run-artifacts/probe_qwen_authority_log.jsonl
The Qwen bank run (real retrieval plus authority)	v2/extractor/experts/gitchameleon/test_qwen_bank.py
GitChameleon base/baked transcripts and solutions (12b, 26b, thinking on/off)	v2/archive/run-artifacts/full-{base,baked}-{12b,26b}*_transcript.jsonl and *_solutions.jsonl
The two-pass recovery run (which 139 misses, recovered 25)	v2/extractor/experts/gitchameleon/recover-26b-think*.jsonl; v2/archive/run-artifacts/*think*
The full Qwen-4B run over 328 and the ladder	v2/archive/run-artifacts/full-qwen-bank_transcript.jsonl; ladder-qwen-bank_solutions.jsonl

13. Publishing and provenance

Claim in the report	Evidence file(s)
The publish playbook, naming scheme, license gemma, factbank.version 0.4.0, delete-to-save-89GB	v2/publish/PUBLISH.md
The 9 per-size model cards for the first three experts (libraries table, mined sources, results)	v2/publish/{security-networking,offensive-security-re,ebpf-dataplane}/gemma-4-<size>-<domain>-expert/README.md
The appsec (fourth expert) publish artifacts and cards	v2/publish/security-appsec/ (see 11.4)
Third-party notices and the code license	THIRD-PARTY-NOTICES.md; LICENSE (repo root)

14. Decision blueprints

Claim in the report	Evidence file(s)
Shipping routes, the size wall being beaten, the 1,911-fact model gates	v2/decisions/FACTBANK-SHIPPING-BLUEPRINT.md
The factbank package plan (sealed loop, delivery options, baked-GGUF rejected)	v2/decisions/PACKAGING-BLUEPRINT.md
The build-nothing database conclusion, embedding cache math	v2/decisions/NEW-DB-RESEARCH.md
Card-mining record and findings (signature mining dead, wording decides)	v2/decisions/CARD-MINING-BLUEPRINT.md; v2/decisions/CARD-MINING-FINDINGS.md
The faceted v3 schema and the thinking-ON enablement ticket	v2/decisions/SCHEMA-V3.md; v2/decisions/TICKET-thinking-on-enablement.md
Jinja engines, the retriever-in-a-template limits	v2/decisions/LANGUAGES.md

15. Harness and script index (runnable evidence)

Script	What it does
v2/extractor/run.py	Runs extract, repair, check for one source into a kept-facts JSONL.
v2/extractor/lookup.py	The Python retrieval test bench: scores facts for queries and prints them (no baking).
v2/bake/template-brain-v3.1/ bake_index.py	Bakes a bank into a GGUF chat-template as an inverted index.
v2/bake/template-brain-v3.1/ render_retrieval.py	Renders a baked template offline against a question and shows the injected facts.
v2/publish/security-appsec/bake6.py	Bakes the appsec bank into six GGUFs (three sizes x thinking-off/thinking-on).
v2/publish/security-appsec/ build_publish.py	Builds and publishes the appsec HF repos + LM Studio Hub virtual models.

v2/bake/serve_factbank.ps1	Launches llama-server with --jinja (and 6-way parallel) for a baked GGUF.
v2/eval/gitchameleon/run_tests.py	Execution scorer: builds pinned venvs and runs hidden pytest for pass@1.
v2/extractor/experts/gitchameleon/ test_baked.py	Drives base vs baked GitChameleon runs over llama-server.
v2/extractor/experts/security-networking/ test_secnet.py	Drives base vs baked security-expert runs (thinking, authority flags).
v2/extractor/experts/gitchameleon/ probe_qwen_authority.py	The controlled authority-framing probe (conditions A/B/C).
v2/bake/template-brain-v3.1/parity.py	Checks jinja2 vs the shipped engine render parity (22/22).

16. Raw run artifacts (the transcripts that were hand-scored)

When a number is described as hand-verified, these are the files that were read. They are committed so the verdict is auditable.

Artifact	Path
Cross-model SecurityEval arms (appsec, all sizes, base and bank) plus DeepSeek-V4	v2/extractor/experts/appsec/benchmark/ se_{e2b,12b,26b}_{base,bank}.jsonl; eval_securityeval_deepseek.jsonl
Security-expert transcripts and scores (netsec, all sizes, base and baked)	v2/extractor/experts/security-networking/ runs/*_transcript.jsonl and *_score.txt
GitChameleon 12b/26b transcripts and solutions	v2/archive/run-artifacts/full-{base,baked}-{12b,26b}*_transcript.jsonl; *_solutions.jsonl
Thinking-on GitChameleon run	v2/archive/run-artifacts/full-baked-12b-think_transcript.jsonl; baked-12b-think_results.jsonl
Authority probe log (the 18 framed rows)	v2/archive/run-artifacts/ probe_qwen_authority_log.jsonl
Qwen-4B full run and difficulty ladder	v2/archive/run-artifacts/full-qwen-bank_transcript.jsonl; ladder-qwen-bank_solutions.jsonl
Two-pass recovery (26b thinking)	v2/extractor/experts/gitchameleon/ recover-26b-think_transcript.jsonl; recover-26b-think_solutions.jsonl
Extraction log	v2/archive/run-artifacts/EXTRACT.log

To prove a single cell in a results table, the path is: open the transcript for that model and condition, find the question id, and read the committed answer against the ground truth. That is the whole method, and it is why the raw transcripts are kept rather than only the scores.